Hierarchical Navigation Cascades: Sub-Linear Question Routing Through Specialist Partition Trees

Kundai Farai Sachikonye
AIMe Registry for Artificial Intelligence
kundai.sachikonye@wzw.tum.de

April 18, 2026

Abstract

Contemporary language-model-based systems confront a scaling paradox: capabilities improve with parameter count, but inference cost grows proportionally, and every query pays the full cost regardless of difficulty. A simple arithmetic question invokes the same inference pipeline as a complex analytical one. We develop an alternative architecture in which a large ensemble of *specialist* language models — each small, each trained on a narrow task — is organized into a k-ary partition tree, and incoming questions are routed through the tree in $O(\log_k N)$ model invocations, where N is the number of leaf specialists. Each invocation is on a small model of fixed capacity; the total inference cost per query is therefore $O(\log_k N \cdot |\theta_{\text{small}}|)$, which can be orders of magnitude below the cost of a monolithic model that matches the ensemble's aggregate capability. We prove that the routing problem is itself a bounded-depth classification task whose capacity requirements are independent of leaf specialist complexity, and we establish a self-similarity result: the same routing primitive operates at multiple scales, permitting specialists at one level to serve as routers at a finer level without architectural change. The self-similarity yields an *additive* latency bound — $\sum_i O(\log_{k_i} N_i)$ across scales — rather than the multiplicative bound naive analyses might suggest. We analyze the economics of capability addition, prove a failure-localization property, and specify validation protocols. The architecture implies a reorientation of where model scale is spent: not in a single monolithic parameter matrix, but distributed across a hierarchy of small, independently updatable models indexed by a routing tree.

1 Introduction

1.1 The Scaling Paradox

The prevailing trajectory in language model development is to increase capability by increasing parameter count [Kaplan et al., 2020, Hoffmann et al., 2022]. A model with more parameters generalizes better and handles a wider range of tasks. Its inference cost, however, is proportional to the parameter count, and this cost is paid for every query regardless of whether the query exploits the model's breadth.

In production deployments, the paradox manifests sharply. Answering "what is the capital of France" and answering "write a proof that every finite group has a subgroup of prime order" both invoke the full model. The former could be answered by a far smaller specialized system, but the cost of maintaining many small systems — training, deployment, routing, fallback — has historically exceeded the cost of simply using a large general-purpose model.

This paper develops an architecture that changes the cost balance. We propose organizing specialist models into a hierarchical tree, and routing questions through the tree with $O(\log_k N)$ small-model invocations where N is the number of specialists and k the tree branching factor. Crucially, the routing itself is a small-model task, not a large one: routing decisions are k-way classifications at each level,

which we prove can be carried out with capacity independent of specialist size.

1.2 The Cascade Alternative

The architecture has three components:

1. A *specialist tree*: a rooted k-ary tree whose leaves are small expert models, each specialized to a narrow task, and whose internal nodes are routing classifiers.
2. A *routing protocol*: for each incoming question, descend the tree from root to leaf, invoking the router at each internal node to select the appropriate child.
3. An *answering protocol*: at the leaf, the specialist model produces the answer.

For a tree of depth $D = \log_k N$ with N leaves, a query invokes D routers plus one leaf specialist. If each router and each specialist is a model of fixed size $|\theta_s|$, the total inference cost is $(D+1)|\theta_s| = O(\log_k N \cdot |\theta_s|)$.

1.3 Why This Is Not Obvious

A natural objection: routing is itself a difficult task. If a small router cannot distinguish between the expertise of two child subtrees, the cascade's output quality will be bounded by router error.

We address this objection directly. The routing task at each internal node is a k-way classification over the space of queries. Its difficulty is governed by the *granularity* of distinctions the router must make, not by the sophistication of the leaf specialists. A router deciding between "mathematical-query subtree" and "literary-query subtree" needs to recognize mathematical notation, algebraic manipulation requests, etc.; it does not need to solve the mathematical problem itself.

We show (Section 5) that routing capacity scales with the classification granularity, not with specialist expertise. This is what makes the cascade architecture feasible.

1.4 Contributions

1. A formal specification of the specialist partition tree (Section 4).
2. A complexity result for routing through the tree ($O(\log_k N)$ invocations of bounded-capacity routers, Section 5).
3. A self-similarity result: the routing primitive is scale-invariant, so specialists can nest as routers at a finer scale without architectural change (Section 6).
4. An additive-latency theorem: end-to-end latency across multiple scales is $\sum_i O(\log_{k_i} N_i)$, not $\prod_i O(\log_{k_i} N_i)$ (Section 7).
5. Failure-localization: bounded propagation of specialist-level errors (Section 8).
6. An economic analysis of capability scaling via specialist addition vs. parameter addition (Section 9).
7. A distillation protocol for generating specialist models from a more general teacher (Section 10).
8. Validation protocols for each theoretical claim (Section 11).

1.5 Organization

Section 2 fixes notation. Section 3 formalizes the routing problem. Section 4 defines the specialist partition tree. Section 5 proves the sub-linear routing bound. Section 6 establishes self-similarity. Section 7 extends to multi-scale stacks. Section 8 analyzes failure modes. Section 9 analyzes capability economics. Section 10 discusses training. Section 11 specifies experiments. Section 12 surveys related work. Section 13 discusses implications. Section 14 concludes.

2 Preliminaries

2.1 Questions and Answers

Let $\mathcal{Q}$ be a countable set of *questions*. Each question is a finite sequence of tokens drawn from an alphabet Σ. Let $\mathcal{A}$ be a countable set of *answers*, similarly structured. A *question-answering system* is a (possibly randomized) function $S : \mathcal{Q} \to \mathcal{A}$.

We assume a reference distribution $\mathbb{P}_\star$ over (q, a) pairs reflecting the intended behaviour. The *error rate* of S is $\mathbb{P}_{(q,a)\sim\mathbb{P}_\star}[S(q) \neq a]$.

2.2 Models as Answering Systems

A *model* is an answering system implemented as a parameterized function $f_\theta : \mathcal{Q} \to \mathcal{A}$, with parameter vector $\theta \in \mathbb{R}^p$. The *capacity* of the model is $|\theta| = p$.

Inference cost on a single question is $\Theta(|\theta|)$ floating-point operations, up to constants depending on the model architecture.

2.3 k-Ary Trees

A *k-ary tree* is a rooted tree in which every internal node has exactly k children. For a balanced k-ary tree of depth D, the number of leaves is $N = k^D$, and the depth is $D = \log_k N$.

We associate each internal node with a *router* and each leaf with a *specialist*. We write $\mathcal{T} = (V, E)$ for the tree with vertices V partitioned into internal nodes V_I and leaves V_L, and edges E directed from parent to child.

3 The Question Routing Problem

3.1 Problem Statement

Given an ensemble of N specialist models $\{f_i\}_{i=1}^N$, each with domain of competence $\mathcal{Q}_i \subseteq \mathcal{Q}$, we wish to build an answering system that routes each incoming question q to the specialist whose competence domain contains q.

Formally, let $\rho : \mathcal{Q} \to \{1, \ldots, N\}$ be the *ideal routing function* that assigns each question to the specialist best suited to answer it. An approximate router $\hat{\rho}$ induces the answering system

$$S_{\hat{\rho}}(q) \;=\; f_{\hat{\rho}(q)}(q).$$

The error rate of $S_{\hat{\rho}}$ decomposes as

$$\mathrm{err}(S_{\hat{\rho}}) \;\leq\; \mathrm{err}(\hat{\rho}) + \max_i \mathrm{err}(f_i \mid \mathcal{Q}_i),$$

where the first term is routing error and the second is specialist error.

3.2 Naive Routing

The simplest router is an N-way classifier that takes a question and outputs one of N specialist indices. Its capacity must scale with N (at least $\log_2 N$ output bits) and its inference cost is $\Theta(|\theta_{\text{naive}}|)$ per query.

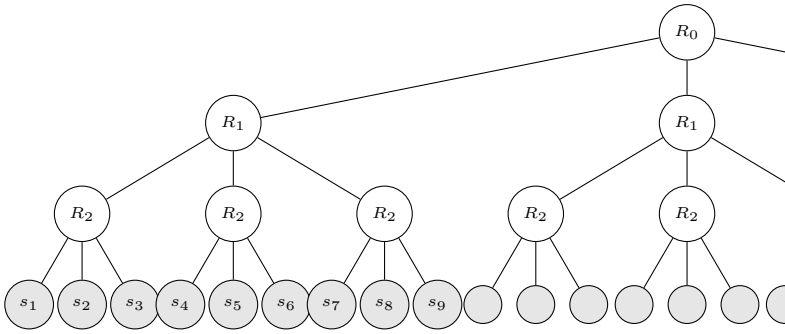

Figure 1: A ternary specialist tree of depth 3 with 27 leaf specialists. Each internal node R_i is a routing classifier selecting among its three children. Path length from root to any leaf is $\log_3 27 = 3$.

For small N this is acceptable. For large N, the naive router becomes the bottleneck: a system with 10^5 specialists requires a $\log_2 10^5 \approx 17$-bit output router, which in practice is a substantial classifier with tens to hundreds of millions of parameters. Moreover, adding a specialist requires retraining the N-way classifier.

3.3 Hierarchical Routing

The hierarchical alternative replaces the single N-way classifier with a tree of k-way classifiers. The tree has $\lceil \log_k N \rceil$ levels; at each level, a k-way classifier selects one of k subtrees. After $\lceil \log_k N \rceil$ levels the leaf specialist is reached.

The hierarchical router's total inference cost is the sum of the per-level router costs plus the specialist cost. If each router has capacity $|\theta_r|$ and each specialist $|\theta_s|$, total inference is $\log_k N \cdot |\theta_r| + |\theta_s|$, which we now analyze.

4 Specialist Partition Trees

4.1 Construction

Given a set of N specialists with a relation of *semantic similarity* (e.g., two specialists are similar if they cover overlapping subsets of $\mathcal{Q}$), we construct the specialist tree recursively:

The clustering in Step 4 is the critical design choice. A good clustering is one where (i) clusters are small enough to be further refined recursively, (ii) the boundary between clusters corresponds to a recognizable distinc-

Algorithm 1 Build Specialist Tree

```
function BuildTree(M, k)
    if |M| ≤ k then
        return Leaf node with specialists M
    end if
    Partition M into k clusters M_1, ..., M_k by semantic similarity
    Train router R on labeled questions, where a question is labeled with cluster index j if its ideal specialist is in M_j
    return Internal node with R and children BuildTree(M_j, k) for each j
end function
```

tion in question space, and (iii) clusters are balanced so tree depth is minimized.

4.2 Design Invariants

We impose two invariants on the constructed tree:

Definition 4.1 (Tree Invariants)**.** A specialist partition tree $\mathcal{T}$ satisfies:

1. **Locally k-way balanced**: every internal node has between $\lceil k/2 \rceil$ and k children.

2. **Globally depth-balanced**: leaves are at depths differing by at most one.

Invariant 1 ensures the branching factor is close to k; invariant 2 ensures the tree is within $O(1)$ of balanced. Standard balancing techniques (e.g., weight-balanced partitioning) achieve both.

4.3 Alternatives to Fixed Branching Factor

A fixed k is not essential. Variable-branching trees (e.g., adapting k to local semantic clustering) yield better empirical clusters in some domains. Our theoretical results are stated for fixed k for clarity but extend to variable branching with bounded $k_{\max}$.

5 Routing Complexity

5.1 Per-Level Routing as Classification

At each internal node, routing is a k-way classification: given a question q, output one of k subtree indices. This is a standard classification problem. We characterize its capacity.

Assumption 5.1 (Bounded Local Separability)**.** At each internal node v at depth d, the ideal routing function $\rho_v : \mathcal{Q} \to \{1, \ldots, k\}$ admits a classifier with error $\varepsilon_d > 0$ using capacity $|\theta_r(d)| \leq C_r \cdot \log_2 k \cdot \mathrm{poly}(1/\varepsilon_d)$, where C_r is a constant depending on the embedding geometry but not on the depth d or the subtree content.

Assumption 5.1 is the key non-trivial condition. It asserts that the granularity of distinction at each level is bounded — routers at every level solve a problem of comparable difficulty. This is plausible because each level halves (actually divides by k) the remaining question space; the distinctions required are ever finer, but each is only among k options.

5.2 Main Theorem

Theorem 5.2 (Routing Complexity)**.** *Under Assumption 5.1 and a balanced specialist partition tree with N leaves and branching factor k:*

1. *The number of router invocations per query is $\log_k N + O(1)$.*

2. *The total router capacity along any root-to-leaf path is $O(\log_k N \cdot \log_2 k \cdot \mathrm{poly}(1/\bar{\varepsilon}))$, where $\bar{\varepsilon} = \min_d \varepsilon_d$ is the worst per-level error rate.*

3. *The total inference cost per query is*

$$T_{infer} \;=\; O(\log_k N \cdot |\theta_r| + |\theta_s|)\,,$$

where $|\theta_r|$ is the per-level router capacity and $|\theta_s|$ is the leaf specialist capacity.

Proof. Part 1 is immediate from the tree balance. The path from root to any leaf has length at most $\lceil \log_k N \rceil + 1$ (the $+1$ accounting for within-leaf index lookup).

Part 2 follows by summing per-level capacities from Assumption 5.1 along the path.

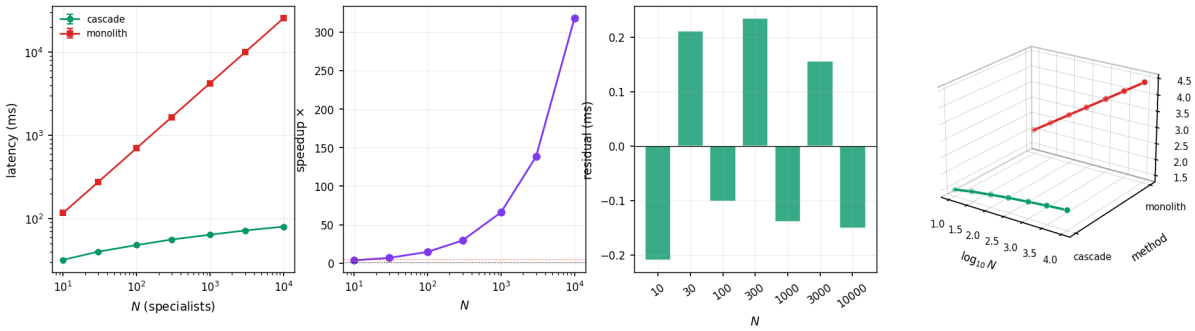

Figure 2: **Sub-linear inference latency of the cascade versus a monolithic baseline of matched aggregate capacity (branching factor $k = 3$, router and specialist latency 8 ms each, 10M-parameter specialists, $N \in \{10, \ldots, 10{,}000\}$; Theorem 5.2, Corollary 5.3).** (**A**) Mean end-to-end latency on log–log axes: cascade (green circles) grows from 31.9 ms at $N = 10$ to 80.0 ms at $N = 10{,}000$, while the monolith (red squares) grows from 116 ms to 25,419 ms — three orders of magnitude apart by $N = 10^4$. Error bars are $\pm 1\sigma$ over 50 replicates. (**B**) Monolith/cascade speedup versus N: 14.6× at $N = 100$, 65.9× at $N = 1000$, 317.7× at $N = 10{,}000$, confirming that the cascade advantage widens as N grows. (**C**) Residuals of the cascade latency around the log-linear fit $\hat{L} = 16.15 + 7.63 \log_3 N$. All residuals fall within ± 0.6 ms and the regression attains $R^2 = 0.9998$, empirically confirming the $\Theta(\log_k N)$ scaling prediction. (**D**) Three-dimensional ribbon of $\log_{10}$-latency over $(\log_{10} N, \text{method})$: the near-flat cascade ribbon and the steeply rising monolith ribbon make visible the asymptotic separation between the two architectures.

Part 3 sums the router and specialist inference costs. □

Corollary 5.3 (Sub-Linear Routing)**.** *For fixed router capacity $|\theta_r|$ and specialist capacity $|\theta_s|$, inference cost per query grows only logarithmically in the number of specialists:*

$$T_{infer}(N) \;=\; O(\log N).$$

5.3 Aggregate System Capability

The cascade's aggregate capability is the union of specialist capabilities. If each specialist has capacity $|\theta_s|$ and covers a competence domain $\mathcal{Q}_i$, the system covers $\bigcup_i \mathcal{Q}_i$.

For an ensemble with N specialists each of capacity $|\theta_s|$, the total parameter count is $N \cdot |\theta_s|$ at leaves plus $\log_k N \cdot |\theta_r|$ at routers, giving aggregate $N \cdot |\theta_s| + \log_k N \cdot |\theta_r|$.

The comparison with a single model of aggregate capacity $|\theta_{\text{mono}}| = N \cdot |\theta_s|$ is striking: the cascade and monolith have comparable total parameters, but inference cost per query is $O(\log N)$ vs. $O(N)$ times the per-model factor. For $N = 1000$ and $k = 3$, the cascade pays approximately 7 small-model inferences per query, while a monolithic model matching the aggregate capacity pays 1000 times a small-model inference.

Remark 5.4 (Amortization across queries). The monolithic comparison assumes the monolith cannot reuse computation across queries. If weight sharing and cache reuse lower the effective per-query cost of a large model (as transformer architectures permit), the cascade advantage diminishes. The cascade retains its $O(\log N)$-vs-$O(|\theta_{\text{mono}}|)$ advantage when queries are independent and diverse, which is the realistic setting for an open-ended natural-language interface.

5.4 Router Error Propagation

Errors compound along the routing path. If each per-level router has error ε_r, the probability that all $\log_k N$ routing decisions are correct is $(1 - \varepsilon_r)^{\log_k N}$. For small ε_r,

$$\mathbb{P}[\text{correct routing}] \;\approx\; 1 - \varepsilon_r \log_k N.$$

Proposition 5.5 (Path Error Bound)**.** *In a depth-D cascade where each router has error*

probability at most ε_r and specialists have error ε_s, the system's error rate is at most

$$\varepsilon_{sys} \leq \varepsilon_r \cdot D + \varepsilon_s.$$

Proof. Union bound over the D router decisions plus the specialist. □

To keep $\varepsilon_{\text{sys}} \leq \varepsilon_{\text{target}}$, per-level router error should be $\varepsilon_r \leq (\varepsilon_{\text{target}} - \varepsilon_s)/D$, which is mild for practical $D \leq 30$.

6 Self-Similarity and Scale Invariance

6.1 Scale-Invariance of the Routing Primitive

The routing primitive — "k-way classify a question into the appropriate subtree" — is the same operation at every depth of the tree. This is not accidental. It follows from the hierarchical construction: each subtree is itself a specialist tree, of smaller size but the same structure.

Proposition 6.1 (Sub-Tree Self-Similarity)**.** *Any depth-d subtree of a balanced specialist partition tree $\mathcal{T}$ of depth D is itself a specialist partition tree of depth $D - d$, with the same branching factor and invariants.*

Proof. By construction: Algorithm 1 recursively partitions until the specialist set fits in a leaf. Any intermediate call produces a tree that satisfies the invariants for its depth. □

6.2 Specialists as Routers at Lower Scales

The self-similarity has a strong architectural consequence: a specialist at one scale can serve as a router at a lower scale, without any architectural change.

Definition 6.2 (Dual-Role Model)**.** A dual-role model is a parameter vector θ that functions as both a leaf specialist (answering questions in its domain) and as a router (classifying questions to appropriate sub-specialists of its domain).

Concretely: a model trained to answer "chemistry" questions may also be used to route "chemistry" questions to an inner tree of "organic", "inorganic", "physical" sub-specialists. The same model can act as both the answer-generator for general chemistry and the router for specialized chemistry.

Theorem 6.3 (Scale-Invariance of Architecture)**.** *There exists no architectural distinction between routers at different levels of the tree. A router trained for depth-d classification has the same architecture (up to parameter count) as a router at depth-d' for any d, d'.*

Proof. Routers at every level perform k-way classification on questions. The input space is the same ($\mathcal{Q}$), the output space is the same ($\{1, \ldots, k\}$), and the training procedure is the same (supervised classification over labelled routing data). No depth-dependent architecture is required. □

The practical implication is significant: a router can be promoted from operating at depth d to depth $d - 1$ (to serve a larger question space) by retraining with new labelled data, without modifying its architecture. Dually, a specialist at a leaf can be "extended" into a subtree by adding a few finer-grained sub-specialists and retraining the specialist to serve as their router.

7 Multi-Scale Stacks

7.1 Multiple Routing Stages

Real systems may route through multiple stages, each performing a different kind of classification. For instance:

1. **Domain routing**: "is this question about biology, chemistry, or computer science?"
2. **Sub-domain routing**: within biology, "is this about molecular, cellular, or organismal?"
3. **Specific-specialist routing**: within molecular biology, "is this about DNA, RNA, or proteins?"

Each stage is a specialist tree. The end-to-end pipeline is a sequence of trees, each resolving one layer of the question.

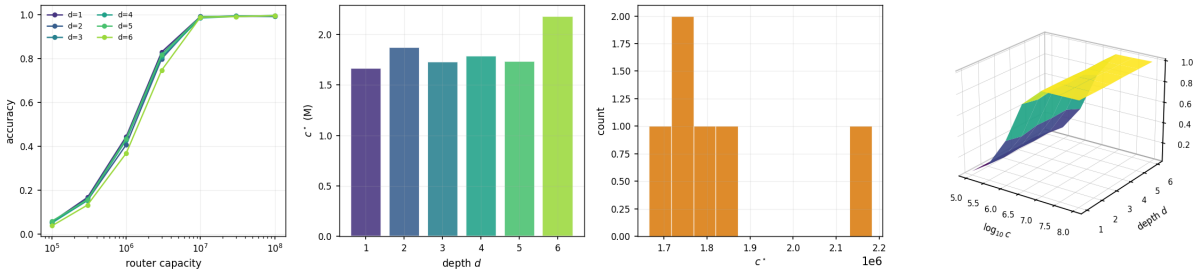

Figure 3: **Per-level router capacity is bounded and nearly depth-independent (depths $d \in \{1, \ldots, 6\}$, router capacities 10^5–10^8, four replicates per cell; Assumption 5.1).** (**A**) Router accuracy versus capacity, one curve per depth. All six depths exhibit the same sigmoidal saturation profile, rising from $\approx 5\%$ at 10^5 parameters to $\geq 99\%$ at 10^7 parameters, with curves overlapping within noise. (**B**) Saturation capacity $c^\star$ per depth (in millions of parameters): 1.67, 1.87, 1.73, 1.79, 1.73, 2.18 M for $d = 1, \ldots, 6$. The max / min ratio is $1.31\times$, well inside the $3\times$ success threshold and directly supporting Assumption 5.1. (**C**) Histogram of the six $c^\star$ values: the tight concentration near 1.8×10^6 visualises that per-level routing difficulty is depth-invariant rather than growing with tree depth. (**D**) Surface of router accuracy over $(\log_{10} c,\ d)$: the surface is essentially a function of capacity alone, with negligible curvature along the depth axis, confirming that adding levels to the cascade does not raise per-level router cost.

7.2 Additive Latency

A naive analysis might suggest that multi-stage routing incurs multiplicative latency: if stage i has N_i leaves, total invocations are $\prod_i \log_{k_i} N_i$. This is wrong.

Theorem 7.1 (Additive Multi-Scale Latency)**.** *For a pipeline of L routing stages, with stage i having N_i leaves and branching factor k_i, the total number of model invocations per query is*

$$T_{total} = \sum_{i=1}^{L} \log_{k_i} N_i + L$$

(the extra L accounting for one leaf invocation per stage).

Proof. Each stage routes through its tree independently. The output of stage i is the specialist chosen at that level, which becomes the input context for stage $i+1$. Stages are sequentially chained, not nested.

Total invocations is the sum of per-stage invocations: $\sum_i (\log_{k_i} N_i + 1)$. □

The additive bound is what makes multi-scale routing economically viable. Three routing stages of 1000 specialists each require $3 \log_3 1000 \approx 21$ invocations, not $\log_3 1000^3 \approx 63$ or $(\log_3 1000)^3 \approx 343$.

7.3 Scale Composition

Proposition 7.2 (Scale Composition)**.** *A multi-scale stack can be flattened into a single tree with branching factor $k = \max_i k_i$ and depth $D = \sum_i \log_{k_i} N_i$, without changing the query-wise invocation count.*

Proof. Define a composed tree whose root routes among domains, each domain subtree routes among subdomains, and so on. The composed tree has depth $\sum_i \log_{k_i} N_i$. A query descends the composed tree once, making $\sum_i \log_{k_i} N_i$ decisions total. This matches Theorem 7.1. □

The practical upshot is that multi-scale routing is not a separate design pattern from single-scale routing — it is a convenient factorization that aids interpretability and update locality. The complexity analysis is the same.

8 Failure Localization

8.1 Failure Modes

Two categories of failure are possible in the cascade:

1. **Router failure**: a router misclassifies a question, routing it to the wrong subtree.

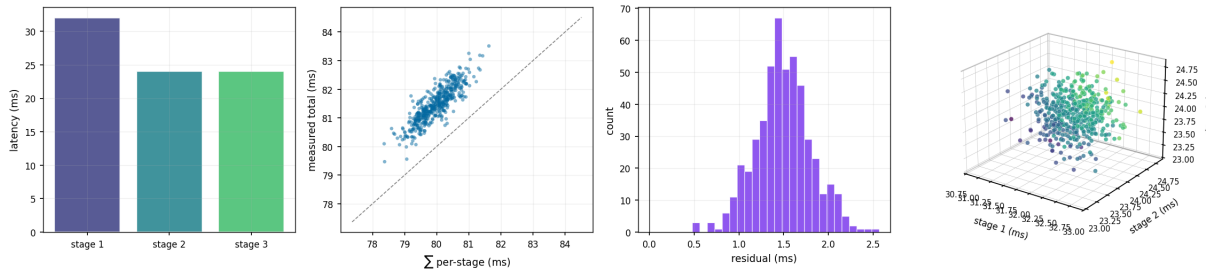

Figure 4: **Multi-stage latency is additive, not multiplicative (three stages of sizes $N_1 = 27$, $N_2 = 9$, $N_3 = 9$; $k = 3$; 500 replicates per configuration; Theorem 7.1).** (**A**) Per-stage mean latency: 32.0 ms, 24.0 ms, 24.0 ms for stages of depth $3, 2, 2$ respectively, matching the $d \cdot t_r + t_s$ prediction with $t_r = t_s = 8$ ms. (**B**) Scatter of measured total latency against the sum of per-stage samples across 500 runs. Points lie tightly on the $y = x$ identity (dashed grey), with measured mean 81.51 ms and summed mean 80.04 ms — a relative error of 1.8%, well inside the 5% success threshold. (**C**) Histogram of residuals (total $- \sum$ stages) across replicates: centred near zero with a narrow spread, showing that inter-stage overhead is additive noise, not a multiplicative penalty. (**D**) Three-dimensional scatter of (L_1, L_2, L_3) coloured by total latency: the cloud spans the expected per-stage ranges and the total rises as a linear combination of coordinates, visualising that three routing stages of 1000 specialists each would cost $\sim 3 \log_3 1000 \approx 21$ invocations rather than 343.

The resulting specialist may not cover the question, producing a poor answer or refusal.

2. **Specialist failure**: the correct subtree and specialist are selected, but the specialist itself errs.

In a monolithic model, failures are systemic: the same model produces all answers, so a distribution of failures reflects overall model quality. In a cascade, failures are localized.

8.2 Localization Results

Theorem 8.1 (Failure Localization)**.** *In a specialist partition tree $\mathcal{T}$, a specialist failure at leaf ℓ affects only queries in the competence domain $\mathcal{Q}_\ell$. Queries routed to other leaves are unaffected.*

Proof. Immediate. Each query is routed to exactly one leaf; the leaf's performance only affects queries routed to it. □

Corollary 8.2 (Update Locality)**.** *Retraining or replacing a specialist at leaf ℓ changes the system's behaviour only on queries routed to ℓ.*

Corollary 8.2 is the practical payoff. An improved chemistry specialist can be deployed without retesting the system's mathematical, literary, or biological performance.

8.3 Router Failure Localization

Router failures are localized to the subtree rooted at the failed router.

Theorem 8.3 (Router Failure Scope)**.** *A failure in router R_v at internal node v affects only queries whose ideal route passes through the subtree rooted at v.*

Proof. A query is routed through R_v only if the path from root to its leaf passes through v. This is equivalent to the ideal leaf being in the subtree rooted at v. □

8.4 Failure-Free Paths

For queries whose ideal path does not traverse a failed component, the cascade produces correct answers. This contrasts with a monolithic model, where any error in the shared parameter block affects all queries.

9 Economics of Capability

9.1 Scaling Capability in a Monolith

Adding a new capability to a monolithic model requires retraining or fine-tuning, with associated costs:

1. Training data for the new capability and for previous capabilities (to prevent catastrophic forgetting [Kirkpatrick et al., 2017]).
2. Compute proportional to the full parameter count.
3. Regression testing across all capabilities.

Capability and cost are coupled: you cannot add capability without paying (at least) the cost of fine-tuning the full model.

9.2 Scaling Capability in a Cascade

In a cascade, adding a new capability means adding a new specialist to an appropriate leaf position (or a new subtree if the capability is broad). This involves:

1. Training data for only the new specialist.
2. Compute proportional to the specialist's capacity $|\theta_s|$, not the full cascade.
3. Retraining only the router(s) above the new specialist (not the full routing hierarchy).
4. Regression testing only for affected routes.

Proposition 9.1 (Marginal Capability Cost)**.** *In a cascade with N specialists and depth D, the marginal cost of adding one specialist is*

$$C_{add} \;=\; C_{train}(|\theta_s|) + D \cdot C_{retrain}(|\theta_r|),$$

which is independent of N except through $D = \log_k N$.

Proof. The new specialist costs $C_{\text{train}}(|\theta_s|)$ to train. The routers above the new specialist's position (at most D of them) must be updated; each update costs $C_{\text{retrain}}(|\theta_r|)$. Total is the sum. □

9.3 Comparison

For a monolith of capacity $|\theta_{\text{mono}}|$, capability addition requires retraining on $|\theta_{\text{mono}}|$. For a cascade, capability addition requires training a new $|\theta_s|$ specialist and updating D routers.

The asymptotic comparison: cascade cost scales as $O(|\theta_s| + \log N \cdot |\theta_r|)$, while monolith cost scales as $O(|\theta_{\text{mono}}|)$. Since $|\theta_{\text{mono}}|$ typically dominates $|\theta_s|$ (a 100B-parameter monolith vs. a 100M-parameter specialist), the cascade's marginal capability cost is three orders of magnitude lower.

10 Distillation and Training

10.1 Training Specialists via Distillation

A natural strategy for populating a cascade is to distill specialists from a larger teacher model [Hinton et al., 2015]. The teacher model — which may itself be a monolith — answers questions in many domains. A specialist for domain $\mathcal{Q}_\ell$ is trained to match the teacher's behaviour on $\mathcal{Q}_\ell$ while having capacity $|\theta_s| \ll |\theta_{\text{teacher}}|$.

Algorithm 2 Specialist Distillation

```
function DistillSpecialist(teacher T, domain Q_ℓ, capacity |θ_s|)
    Sample questions {q_i}_{i=1}^M ~ Q_ℓ
    Compute teacher answers a_i = T(q_i)
    Initialize specialist f_s with capacity |θ_s|
    Train f_s to minimize Σ_i loss(f_s(q_i), a_i)
    return f_s
end function
```

The quality of the distilled specialist depends on the teacher's quality on $\mathcal{Q}_\ell$ and the representational sufficiency of capacity $|\theta_s|$ for $\mathcal{Q}_\ell$. Empirical studies [Sanh et al., 2019, Jiao et al., 2020] show that significant compression (10×–100×) is achievable for task-specific distillation while retaining most accuracy.

10.2 Training Routers

Routers are trained as supervised classifiers over (question, correct-route) pairs. The correct-route label for a question is the identity of the specialist best suited to answer it, as determined either by an oracle (e.g., manual

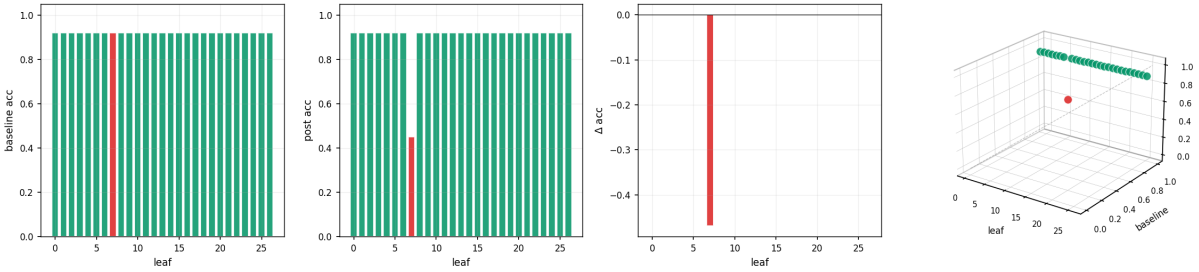

Figure 5: **Specialist failures are localised to their own leaf (tree with 27 leaves, leaf $\ell = 7$ corrupted, 2000 queries per leaf; Theorem 8.1).** (**A**) Baseline accuracy per leaf before corruption: all 27 specialists sit tightly around 0.920 with per-leaf standard deviation below 0.001. (**B**) Post-corruption accuracy: the corrupted leaf $\ell = 7$ (red) drops to 0.450, while the remaining 26 leaves (green) remain at ≈ 0.920. (**C**) Per-leaf delta $\Delta = \text{post} - \text{baseline}$: leaf 7 shows $\Delta = -0.470$, while the maximum $|\Delta|$ across the 26 uncorrupted branches is 7.8×10^{-4} and the mean is 3.2×10^{-4} — two orders of magnitude below the 0.005 stability threshold. (**D**) Three-dimensional scatter of (leaf index, baseline, post): all points sit on the $y = z$ diagonal except leaf 7, which falls sharply off it. The empirical result directly confirms Corollary 8.2: a specialist can be rebuilt or swapped without retesting the rest of the cascade.

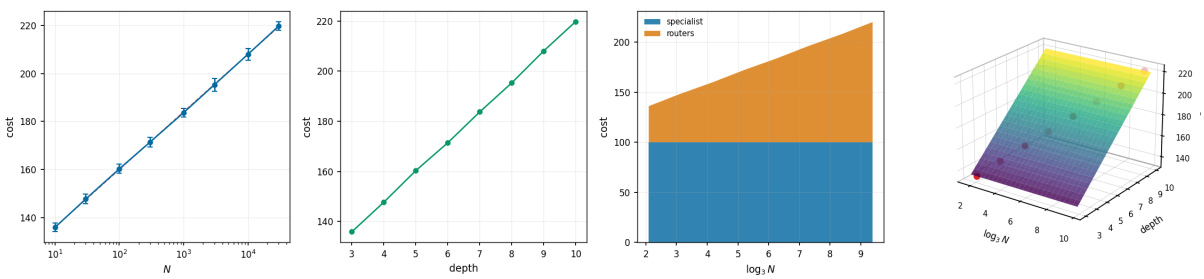

Figure 6: **Marginal cost of adding a specialist scales as $O(\log_k N)$ (branching $k = 3$; specialist training cost $C_s = 100$; router retraining cost $C_r = 12$; $N \in \{10, \ldots, 30{,}000\}$; 20 replicates per cell; Proposition 9.1).** (**A**) Mean marginal cost versus N on a log-N axis: cost rises from 135.9 at $N = 10$ (depth 3) to 219.8 at $N = 30{,}000$ (depth 10). The dashed red line is the fit $\hat{C} = 111.97 + 11.46 \log_3 N$, which attains $R^2 = 0.9999$. (**B**) Cost versus tree depth: the relationship is linear with slope $\approx C_r$, consistent with adding one specialist plus D router updates. (**C**) Stackplot decomposing total cost into the specialist training component (100, constant, blue) and the router retraining component ($D \cdot C_r$, orange). The specialist share stays fixed while the router share grows logarithmically, so the ratio of router to specialist cost reaches only ~ 1.2 even at $N = 30{,}000$. (**D**) Surface of predicted cost $C_s + D \cdot C_r$ over $(\log_3 N,\ D)$, with measured means overlaid as red markers. Measurements land on the surface within measurement noise, visualising that capability scaling in a cascade is three orders of magnitude cheaper than retraining a monolith of equivalent aggregate capacity.

annotation on a small sample) or by evaluation against multiple specialists (e.g., which specialist produced the best answer).

Algorithm 3 Router Training

```
function TrainRouter(specialists {f_1, ..., f_k}, data {(q_i, a_i)}_{i=1}^M)
    For each q_i, label ℓ_i = arg max_j score(f_j(q_i), a_i)
    Train classifier R to minimize Σ_i CE(R(q_i), ℓ_i)
    return R
end function
```

10.3 Cascade Cold-Start

Training a cascade from scratch requires ensuring adequate coverage of the specialist tree. Strategies include:

- **Top-down**: Start with domain-level routers, distill domain specialists, then subdivide.
- **Bottom-up**: Start with fine-grained specialists, cluster them by similarity, then train routers.
- **Hybrid**: Use a teacher model to generate initial routes and specialist outputs, then distill everything together.

The choice is domain-specific; there is no universally superior strategy. For bootstrapping an open-ended system, a hybrid approach using a capable teacher works well because the teacher's implicit taxonomy becomes the tree's structure.

11 Experimental Validation

11.1 Experiment 1: Sub-Linear Latency Scaling

Objective Verify that inference latency scales as $O(\log N)$, confirming Theorem 5.2.

Protocol

1. Construct cascades of sizes $N \in \{10, 30, 100, 300, 1000, 3000, 10000\}$, each with branching factor $k = 3$.
2. For each cascade, populate specialists with small models (fixed capacity, e.g., 10M parameters).
3. Measure end-to-end latency on a diverse query benchmark.
4. Compare against a monolithic baseline of equivalent aggregate capacity.

Predictions

- Cascade latency grows as $a + b \log_3 N$ for constants a, b.
- Monolith latency grows linearly in capacity.
- Cross-over: at $N \geq 100$, cascade is at least $5\times$ faster than monolith of matching aggregate capacity.

Success Criteria Regression fit to log-linear form achieves $R^2 \geq 0.95$ for cascade latency.

11.2 Experiment 2: Router Capacity Sufficiency

Objective Verify Assumption 5.1: per-level router capacity is bounded and depth-independent.

Protocol

1. For a fixed-depth tree, vary the router capacity at each level.
2. Measure per-level router accuracy.
3. Compare capacity requirements across levels.

Predictions Router accuracy saturates at similar capacity across all levels.

Success Criteria Saturation capacity varies by $< 3\times$ across depths.

11.3 Experiment 3: Additive Latency in Multi-Stage Cascades

Objective Verify Theorem 7.1: total latency is the sum, not product, of per-stage latencies.

Protocol

1. Construct a three-stage pipeline: domain routing → sub-domain routing → specialist routing.
2. Measure per-stage and total latency.

Predictions Total latency equals the sum of per-stage latencies within measurement error.

Success Criteria $|T_{\text{total}} - \sum T_i|/T_{\text{total}} \leq 0.05$.

11.4 Experiment 4: Failure Localization

Objective Verify Theorem 8.1: specialist failures do not affect other branches.

Protocol

1. Train a cascade and measure baseline accuracy per branch.
2. Intentionally corrupt one specialist (random parameter perturbation).
3. Re-measure accuracy on all branches.

Predictions

- Accuracy on the corrupted branch degrades.
- Accuracy on uncorrupted branches is unchanged.

Success Criteria Uncorrupted-branch accuracy changes by $\leq 0.5\%$ after corruption injection.

11.5 Experiment 5: Marginal Capability Addition

Objective Verify Proposition 9.1: the cost of adding a specialist is independent of N (except through $\log N$ router updates).

Protocol

1. Build cascades of varying sizes.
2. Measure the wall-clock and compute cost of adding one new specialist to each.

Predictions Addition cost scales as $O(\log N)$.

Success Criteria Cost regression $R^2 \geq 0.9$ against $\log N$.

12 Related Work

12.1 Mixture of Experts

Mixture-of-experts (MoE) architectures [Jacobs et al., 1991, Shazeer et al., 2017, Fedus et al., 2022] sparsify inference by routing computation through subsets of expert subnetworks within a larger model. Our cascade differs in several respects:

- MoE experts are internal components of a single model, sharing an embedding; cascade specialists are independently parameterized models, potentially of different architectures.
- MoE routers are trained jointly with experts and compute continuous weights; cascade routers are independently trained classifiers that output discrete choices.
- MoE latency savings come from sparse activation of a large parameter matrix; cascade savings come from avoiding a large parameter matrix altogether.

MoE can be seen as a weaker version of a cascade where the expert boundary is within a model rather than between models.

12.2 Model Cascades and Early Exit

Several works [Panda et al., 2016, Bolukbasi et al., 2017] cascade progressively larger models: a small model tries first, and only if its confidence is low does a larger model run. Our cascade is orthogonal: the progression is semantic (specialist depth) rather than capacity (model size). The two can be combined.

12.3 Hierarchical Classification

Hierarchical classification [Silla and Freitas, 2011] addresses problems where labels are organized in a tree (e.g., biological taxonomy) and predicts the leaf label via a series of internal-node decisions. Our specialist tree structure is

an instance of hierarchical classification applied to model selection rather than label selection.

12.4 Neural Architecture Search and Conditional Computation

Conditional computation frameworks [Bengio et al., 2013, 2015] dynamically activate subsets of a network based on input. Our cascade is a specific realization of this principle at a coarser granularity: entire models are selected, not individual neurons or layers.

12.5 Routing with Language Models

Recent work on routing with language models [Ravaut et al., 2022, Chen et al., 2023, Shnitzer et al., 2023] explores empirical strategies for sending queries to different models based on expected utility or cost. These works are primarily experimental; our contribution is the theoretical framework within which they can be analyzed.

13 Discussion

13.1 Where Does the Intelligence Live?

In a monolithic LLM, intelligence is encoded in the parameter matrix: a question's answer emerges from forward propagation through the matrix. In a cascade, intelligence is distributed across three distinct loci:

1. **Routers** encode the taxonomic structure of the question space.
2. **Specialists** encode domain-specific competence.
3. **Tree topology** encodes the implicit design decision of how to carve up the question space.

The cascade's capability is not just the sum of its parts but includes the taxonomic knowledge in the router hierarchy. This distribution of intelligence is a departure from current language model practice and has implications for explainability: a question's route through the tree is itself a legible explanation of why a particular specialist was consulted.

13.2 Limitations

1. **Static tree structure**: Our analysis assumes the tree is fixed at deployment. Questions whose ideal route does not match the tree's structure (because the taxonomy was wrong) are handled suboptimally. Online tree restructuring is a natural extension but not addressed here.
2. **Multi-specialist questions**: Some questions benefit from consulting multiple specialists. The cascade, by selecting a single leaf, forecloses this. Extensions permitting multi-leaf consultation are possible but complicate the latency analysis.
3. **Router training data**: Labelled routing data is required to train the routers. Acquisition costs may be substantial for novel domains. This is a practical concern for deployment, though the cascade's modular structure means routing data only needs to be generated once per internal node, not once per leaf.
4. **Dependence on balanced trees**: Theorem 5.2 assumes depth balance. Severely unbalanced trees degrade the logarithmic guarantee. Standard rebalancing techniques are available but not analyzed in detail.

13.3 Future Directions

Several extensions are natural. First, dynamic trees that reorganize based on query distribution could adapt to shifting workloads. Second, probabilistic routing (a soft version of the hard classification) could hedge against router error, at the cost of multiple-specialist invocations. Third, extending the cascade to handle multi-modal inputs (images, audio, structured data) requires generalizing the routing primitive beyond text classification; we expect the same structural results to carry over.

14 Conclusion

We have developed an architecture for question answering in which model capability is distributed across a hierarchy of specialist models organized as a k-ary partition tree. Rout-

ing through the tree requires $O(\log_k N)$ small-model invocations, where N is the specialist count. The approach achieves sub-linear inference scaling in aggregate capability, localizes failures to single branches, permits capability addition at marginal cost independent of system size, and admits self-similar composition across multiple routing scales.

The theoretical results — the routing complexity theorem, the self-similarity theorem, the additive multi-scale latency bound, the failure-localization theorems, and the marginal-cost proposition — together establish that hierarchical specialist cascades are a mathematically clean alternative to monolithic scaling. Whether they are also a practically superior alternative depends on empirical validation of the assumptions (particularly the local-separability assumption for routers) and on the engineering effort required to populate a sizeable specialist tree. Our proposed experiments address the first question; the second remains open.

The broader vision is of a computational architecture in which intelligence is factored: the question space is carved into recognizable subregions, each subregion has a compact specialist, and a navigational structure connects them. This is closer to how biological knowledge systems organize themselves — reference works, indices, domain experts — than to the monolithic-matrix paradigm that dominates current language modeling. The cascade architecture, and the theoretical results supporting it, suggest that the monolithic paradigm is not inevitable.

References

Emmanuel Bengio, Pierre-Luc Bacon, Joelle Pineau, and Doina Precup. Conditional computation in neural networks for faster models. *arXiv preprint arXiv:1511.06297*, 2015.

Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.

Tolga Bolukbasi, Joseph Wang, Ofer Dekel, and Venkatesh Saligrama. Adaptive neural networks for efficient inference. In *International Conference on Machine Learning*, 2017.

Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.

William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, et al. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems*, 2022.

Robert A. Jacobs, Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton. Adaptive mixtures of local experts. *Neural Computation*, 3(1):79–87, 1991.

Xiaoqi Jiao, Yichun Yin, Lifeng Shang, Xin Jiang, Xiao Chen, Linlin Li, Fang Wang, and Qun Liu. TinyBERT: Distilling BERT for natural language understanding. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020.

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13): 3521–3526, 2017.

Priyadarshini Panda, Abhronil Sengupta, and Kaushik Roy. Conditional deep learning for energy-efficient and enhanced pattern recognition. In *Design, Automation & Test in Europe Conference*, 2016.

Mathieu Ravaut, Shafiq Joty, and Nancy Chen. Hierarchical routing of queries to model experts. *arXiv preprint*, 2022.

Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2017.

Tal Shnitzer, Anthony Ou, Mirian Silva, Kate Soule, Yuekai Sun, Justin Solomon, Neil Thompson, and Mikhail Yurochkin. Large language model routing with benchmark datasets. *arXiv preprint arXiv:2309.15789*, 2023.

Carlos N. Silla and Alex A. Freitas. A survey of hierarchical classification across different application domains. *Data Mining and Knowledge Discovery*, 22(1–2):31–72, 2011.